

Note!

3NF loading procedure (the procedure intentionally splitted to several screenshots to add explanations)

3NF layer

Precondition:

Indexes are created (to accelerate search with watermark read(explained further))

```
CREATE INDEX IF NOT EXISTS idx_ce_orders_update_dt ON bl_3nf.ce_orders (update_dt);  
CREATE INDEX IF NOT EXISTS idx_ce_orders_order_dt ON bl_3nf.ce_orders (order_dt);
```

Declared three counters for tracking rows affected, a temp variable for accumulating GET DIAGNOSTICS counts, and v_watermark - the timestamp variable marking where the last successful run left off(compares date with the last in the log table)

```
CREATE OR REPLACE PROCEDURE bl_cl.load_ce_orders()  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_rows_ins INT = 0;  
    v_rows_upd INT = 0;  
    v_tmp      INT = 0;  
    v_watermark TIMESTAMP;  
BEGIN  
    SELECT COALESCE(MAX(log_dt), TIMESTAMP '1900-01-01')  
    INTO v_watermark  
    FROM bl_cl.mta_load_logs  
    WHERE procedure_name = 'load_ce_orders' AND log_status = 'SUCCESS';
```

INT incoming CTE.

Pulls raw rows from the INT staging table. The crucial line is insert_date > v_watermark - this is the incremental filter that limits the scan to only staging rows added since the last run. Everything older was already loaded, so it's skipped.

```

-- INT source
WITH incoming AS (
    SELECT
        COALESCE(NULLIF(o.order_id_int, ''), 'n. a.') AS order_src_id,
        CAST(COALESCE(NULLIF(o.order_dt_int, ''), '1900-01-01') AS DATE) AS order_dt,
        COALESCE(NULLIF(o.order_status_int, ''), 'n. a.') AS order_status,
        COALESCE(NULLIF(o.returned, ''), 'n. a.') AS returned,
        COALESCE(NULLIF(o.customer_id_int, ''), 'n. a.') AS customer_src_id,
        COALESCE(NULLIF(o.sales_rep_id_int, ''), 'n. a.') AS rep_src_id,
        COALESCE(NULLIF(o.product_id_int, ''), 'n. a.') AS product_src_id,
        COALESCE(NULLIF(o.subchannel, ''), 'n. a.') AS subchannel_src_id,
        COALESCE(NULLIF(o.city_int, ''), 'n. a.') AS city_src_id,
        COALESCE(NULLIF(o.payment, ''), 'n. a.') AS payment_method,
        CAST(COALESCE(NULLIF(o.price, ''), '0') AS NUMERIC(10,2)) AS unit_price,
        CAST(COALESCE(NULLIF(o.quantity_int, ''), '0') AS BIGINT) AS quantity,
        CAST(COALESCE(NULLIF(o.discount_percent_int, ''), '0') AS BIGINT) AS discount_percent,
        CAST(COALESCE(NULLIF(o.discount_amount_int, ''), '0') AS NUMERIC(10,2)) AS discount_amount,
        CAST(COALESCE(NULLIF(o.shipping_cost_int, ''), '0') AS NUMERIC(10,2)) AS shipping_cost,
        CAST(COALESCE(NULLIF(o.tax, ''), '0') AS NUMERIC(10,2)) AS tax_amount,
        CAST(COALESCE(NULLIF(o.order_amount_int, ''), '0') AS NUMERIC(10,2)) AS order_amount,
        CAST(COALESCE(NULLIF(o.cost_amount_int, ''), '0') AS NUMERIC(10,2)) AS cost_amount,
        CAST(COALESCE(NULLIF(o.pprofit_margin_percent_int, ''), '0') AS NUMERIC(10,2)) AS profit_margin_percent,
        CAST(COALESCE(NULLIF(o.profit_amount_int, ''), '0') AS NUMERIC(10,2)) AS profit_amount
    FROM sa_int_sales.src_int_sales o
    WHERE o.order_id_int IS NOT NULL
    AND o.insert_date > v_watermark
),

```

INT mapped CTE.

Takes each cleaned order and resolves its natural keys into 3NF surrogate ids by LEFT JOINing to each dimension table (products, employees, customers, etc.). COALESCE(..., -1) sends any unmatched key to the -1 default row so the foreign keys never fail. The customer join adds order_dt BETWEEN start_dt AND end_dt to pick the SCD2 version that was active on the order's date.

```

mapped AS (
    SELECT s.*,
        COALESCE(c.customer_id, -1) AS customer_id,
        COALESCE(e.sales_rep_id, -1) AS sales_rep_id,
        COALESCE(p.product_id, -1) AS product_id,
        COALESCE(sc.subchannel_id, -1) AS subchannel_id,
        COALESCE(ci.city_id, -1) AS city_id
    FROM incoming s
    LEFT JOIN bl_3nf.ce_customers_scd c
        ON c.customer_src_id = s.customer_src_id
        AND c.source_system = 'SA_INT_SALES'
        AND c.is_active = 'Y'
    LEFT JOIN bl_3nf.ce_employees e
        ON e.sales_rep_src_id = s.rep_src_id AND e.source_system = 'SA_INT_SALES'
    LEFT JOIN bl_3nf.ce_products p
        ON p.product_src_id = s.product_src_id AND p.source_system = 'SA_INT_SALES'
    LEFT JOIN bl_3nf.ce_subchannels sc
        ON sc.subchannel_src_id = s.subchannel_src_id AND sc.source_system = 'SA_INT_SALES'
    LEFT JOIN bl_3nf.ce_cities ci
        ON ci.city_name_src_id = s.city_src_id AND ci.source_system = 'SA_INT_SALES'
)

```

INT INSERT ... ON CONFLICT

Inserts the mapped orders into ce_orders, assigning a fresh surrogate via nextval. ON CONFLICT (order_src_id, source_system) DO UPDATE makes it idempotent a re-encountered order updates only if its status/amount actually changed, otherwise it's left alone. GET DIAGNOSTICS captures the affected count into the running total.

```

)
INSERT INTO bl_3nf.ce_orders (
  order_id, order_dt, order_status, returned, customer_id, sales_rep_id,
  product_id, subchannel_id, city_id, payment_method, unit_price, quantity,
  discount_percent, discount_amount, shipping_cost, tax_amount, order_amount,
  cost_amount, profit_margin_percent, profit_amount, transaction_id,
  source_system, source_entity, order_src_id, insert_dt, update_dt)
SELECT nextval('bl_3nf.seq_orders_id'),
  m.order_dt, m.order_status, m.returned, m.customer_id, m.sales_rep_id,
  m.product_id, m.subchannel_id, m.city_id, m.payment_method, m.unit_price,
  m.quantity, m.discount_percent, m.discount_amount, m.shipping_cost,
  m.tax_amount, m.order_amount, m.cost_amount, m.profit_margin_percent,
  m.profit_amount, m.order_src_id,
  'SA_INT_SALES', 'SRC_INT_SALES', m.order_src_id, CURRENT_DATE, CURRENT_DATE
FROM mapped m
ON CONFLICT (order_src_id, source_system) DO UPDATE
  SET order_status = EXCLUDED.order_status,
      returned      = EXCLUDED.returned,
      order_amount  = EXCLUDED.order_amount,
      profit_amount  = EXCLUDED.profit_amount,
      update_dt     = CURRENT_DATE
  WHERE ce_orders.order_status IS DISTINCT FROM EXCLUDED.order_status
     OR ce_orders.returned     IS DISTINCT FROM EXCLUDED.returned
     OR ce_orders.order_amount IS DISTINCT FROM EXCLUDED.order_amount
     OR ce_orders.profit_amount IS DISTINCT FROM EXCLUDED.profit_amount;
GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins = v_rows_ins + v_tmp;

```

US incoming / mapped / INSERT blocks

The same three-stage pattern repeated for the US source, with the differences being the US column names, the SA_US_SALES source_system, and one extra join: US cities are matched through their state (ci.state_id = st.state_id) because the same city name exists in multiple states. Counts accumulate into the same variables as the INT block.

```

WITH incoming AS (
    SELECT
        COALESCE(NULLIF(o.order_id, ''), 'n. a.') AS order_src_id,
        CAST(COALESCE(NULLIF(o.order_dt, ''), '1900-01-01') AS DATE) AS order_dt,
        COALESCE(NULLIF(o.order_status, ''), 'n. a.') AS order_status,
        COALESCE(NULLIF(o.returned, ''), 'n. a.') AS returned,
        COALESCE(NULLIF(o.customer_id, ''), 'n. a.') AS customer_src_id,
        COALESCE(NULLIF(o.sales_rep_id, ''), 'n. a.') AS rep_src_id,
        COALESCE(NULLIF(o.product_id, ''), 'n. a.') AS product_src_id,
        COALESCE(NULLIF(o.subchannel, ''), 'n. a.') AS subchannel_src_id,
        COALESCE(NULLIF(o.city, ''), 'n. a.') AS city_src_id,
        COALESCE(NULLIF(o.state, ''), 'n. a.') AS state_src_id,
        COALESCE(NULLIF(o.payment_method, ''), 'n. a.') AS payment_method,
        CAST(COALESCE(NULLIF(o.unit_price, ''), '0') AS NUMERIC(10,2)) AS unit_price,
        CAST(COALESCE(NULLIF(o.quantity, ''), '0') AS BIGINT) AS quantity,
        CAST(COALESCE(NULLIF(o.discount_percent, ''), '0') AS BIGINT) AS discount_percent,
        CAST(COALESCE(NULLIF(o.discount_amount, ''), '0') AS NUMERIC(10,2)) AS discount_amount,
        CAST(COALESCE(NULLIF(o.shipping_cost, ''), '0') AS NUMERIC(10,2)) AS shipping_cost,
        CAST(COALESCE(NULLIF(o.tax_amount, ''), '0') AS NUMERIC(10,2)) AS tax_amount,
        CAST(COALESCE(NULLIF(o.order_amount, ''), '0') AS NUMERIC(10,2)) AS order_amount,
        CAST(COALESCE(NULLIF(o.cost_amount, ''), '0') AS NUMERIC(10,2)) AS cost_amount,
        CAST(COALESCE(NULLIF(o.profit_margin_percent, ''), '0') AS NUMERIC(10,2)) AS profit_margin_percent,
        CAST(COALESCE(NULLIF(o.profit_margin_amount, ''), '0') AS NUMERIC(10,2)) AS profit_amount
    FROM sa_us_sales.src_us_sales o
    WHERE o.order_id IS NOT NULL
        AND o.insert_date > v_watermark
),
mapped AS (
    SELECT s.*,
        COALESCE(c.customer_id, -1) AS customer_id,
        COALESCE(e.sales_rep_id, -1) AS sales_rep_id,
        COALESCE(p.product_id, -1) AS product_id,
        COALESCE(sc.subchannel_id, -1) AS subchannel_id,
        COALESCE(ci.city_id, -1) AS city_id
    FROM incoming s
    LEFT JOIN bl_3nf.ce_customers_scd c
        ON c.customer_src_id = s.customer_src_id
        AND c.source_system = 'SA_US_SALES'
        AND c.is_active = 'Y'
    LEFT JOIN bl_3nf.ce_employees e
        ON e.sales_rep_src_id = s.rep_src_id AND e.source_system = 'SA_US_SALES'
    LEFT JOIN bl_3nf.ce_products p
        ON p.product_src_id = s.product_src_id AND p.source_system = 'SA_US_SALES'
    LEFT JOIN bl_3nf.ce_subchannels sc
        ON sc.subchannel_src_id = s.subchannel_src_id AND sc.source_system = 'SA_US_SALES'
    LEFT JOIN bl_3nf.ce_states st
        ON st.state_name_src_id = s.state_src_id AND st.source_system = 'SA_US_SALES'
    LEFT JOIN bl_3nf.ce_cities ci
        ON ci.city_name_src_id = s.city_src_id
        AND ci.source_system = 'SA_US_SALES'
        AND ci.state_id = st.state_id
)
INSERT INTO bl_3nf.ce_orders (
    order_id, order_dt, order_status, returned, customer_id, sales_rep_id,
    product_id, subchannel_id, city_id, payment_method, unit_price, quantity,
    discount_percent, discount_amount, shipping_cost, tax_amount, order_amount,
    cost_amount, profit_margin_percent, profit_amount, transaction_id,
    source_system, source_entity, order_src_id, insert_dt, update_dt)
SELECT nextval('bl_3nf.seq_orders_id'),
    m.order_dt, m.order_status, m.returned, m.customer_id, m.sales_rep_id,
    m.product_id, m.subchannel_id, m.city_id, m.payment_method, m.unit_price,
    m.quantity, m.discount_percent, m.discount_amount, m.shipping_cost,
    m.tax_amount, m.order_amount, m.cost_amount, m.profit_margin_percent,
    m.profit_amount, m.order_src_id,
    'SA_US_SALES', 'SRC_US_SALES', m.order_src_id, CURRENT_DATE, CURRENT_DATE
FROM mapped m
ON CONFLICT (order_src_id, source_system) DO UPDATE
    SET customer_id = EXCLUDED.customer_id,
        order_status = EXCLUDED.order_status,
        returned = EXCLUDED.returned,
        order_amount = EXCLUDED.order_amount,
        profit_amount = EXCLUDED.profit_amount,
        update_dt = CURRENT_DATE;
GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins = v_rows_ins + v_tmp;

```

```

INSERT INTO bl_3nf.ce_orders (
  order_id, order_dt, order_status, returned, customer_id, sales_rep_id,
  product_id, subchannel_id, city_id, payment_method, unit_price, quantity,
  discount_percent, discount_amount, shipping_cost, tax_amount, order_amount,
  cost_amount, profit_margin_percent, profit_amount, transaction_id,
  source_system, source_entity, order_src_id, insert_dt, update_dt)
SELECT nextval('bl_3nf.seq_orders_id'),
  m.order_dt, m.order_status, m.returned, m.customer_id, m.sales_rep_id,
  m.product_id, m.subchannel_id, m.city_id, m.payment_method, m.unit_price,
  m.quantity, m.discount_percent, m.discount_amount, m.shipping_cost,
  m.tax_amount, m.order_amount, m.cost_amount, m.profit_margin_percent,
  m.profit_amount, m.order_src_id,
  'SA_US_SALES', 'SRC_US_SALES', m.order_src_id, CURRENT_DATE, CURRENT_DATE
FROM mapped m
ON CONFLICT (order_src_id, source_system) DO UPDATE
  SET order_status = EXCLUDED.order_status,
  returned = EXCLUDED.returned,
  order_amount = EXCLUDED.order_amount,
  profit_amount = EXCLUDED.profit_amount,
  update_dt = CURRENT_DATE
  WHERE ce_orders.order_status IS DISTINCT FROM EXCLUDED.order_status
  OR ce_orders.returned IS DISTINCT FROM EXCLUDED.returned
  OR ce_orders.order_amount IS DISTINCT FROM EXCLUDED.order_amount
  OR ce_orders.profit_amount IS DISTINCT FROM EXCLUDED.profit_amount;
GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins = v_rows_ins + v_tmp;

```

Success log

After both sources load without error, calls p_log with 'SUCCESS', the total rows affected, and a message noting the watermark used. This log row's timestamp becomes the next run's watermark — closing the incremental loop.

EXCEPTION block

If anything in the procedure errors, control jumps here, and p_log records an 'ERROR' row with SQLERRM (the error text). Because the error is caught, the log write survives instead of rolling back and note that on error no SUCCESS row is written, so the next run reuses the previous watermark and safely retries the same data.

```

CALL bl_cl.p_log('load_ce_orders', 'SUCCESS', v_rows_ins,
  'Rows affected: ' || v_rows_ins ||
  ' (incremental since ' || v_watermark || ')');
EXCEPTION WHEN OTHERS THEN
  CALL bl_cl.p_log('load_ce_orders', 'ERROR', NULL, SQLERRM);
END;
$$;

```

TEST

```

CALL bl_cl.load_ce_orders();
SELECT * FROM bl_cl.mta_load_logs WHERE procedure_name='load_ce_orders' ORDER BY log_id DESC;

```

ld_logs 1mta_load_logs 2ce_orders 2 (2)

* FROM bl_cl.mta_load_logs WHERE procedure_name=Data filter is not supported

log_id	log_dt	AZ procedure_name	AZ log_status	123 rows_affected	AZ log_message
124	2026-07-25 13:20:56.886	load_ce_orders	SUCCESS	0	Rows affected: 0 (incremental since 2026-07-24 01:01:01.770121)
85	2026-07-24 01:01:01.770	load_ce_orders	SUCCESS	0	Rows affected: 0 (incremental since 2026-07-24 00:58:47.676052)
84	2026-07-24 00:58:47.676	load_ce_orders	SUCCESS	1,000,000	Rows affected: 1000000 (incremental since 1900-01-01 00:00:00)

```
SELECT * FROM bl_dm.fct_orders ORDER BY order_id LIMIT 500;
```

123 order_id	order_dt	AZ order_status	AZ returned	123 customer_id	123 sales_rep_id	123 product_id	123 subchannel_id	123 city_id	AZ payment_meth
2,000,100	2025-02-05	Delivered	No	381,072	163	377	102	823	PayPal
2,000,101	2023-02-18	Delivered	No	418,594	101	1,561	102	816	Debit Card
2,000,102	2024-04-02	Shipped	No	419,872	116	1,885	101	793	PayPal
2,000,103	2023-02-25	Delivered	No	398,037	154	1,292	102	831	PayPal
2,000,104	2024-04-25	Delivered	No	412,066	105	2,052	102	797	PayPal
2,000,105	2025-05-01	Delivered	No	367,209	185	1,201	103	805	Debit Card
2,000,106	2023-12-10	Shipped	No	389,557	182	1,142	103	788	Cash on Delivery
2,000,107	2024-07-20	Processing	No	323,329	146	2,072	102	827	Bank Transfer
2,000,108	2024-01-23	Returned	Yes	337,343	125	2,169	104	824	Debit Card
2,000,109	2023-11-06	Delivered	No	417,799	157	1,527	101	809	Cash on Delivery
2,000,110	2024-03-07	Processing	No	377,468	176	2,284	100	829	PayPal
2,000,111	2026-04-18	Shipped	No	302,111	105	397	102	797	Bank Transfer
2,000,112	2024-12-12	Processing	No	370,767	176	1,797	103	803	Debit Card

FACT TABLE PROCESS

Precondition:

Fact table created with PARTITION BY RANGE (event_dt) CONSTRAINT

PROCEDURE bl_cl.load_fct_orders_dd

The procedure accepts an input date (p_month_start) and automatically calculates the upper date boundary (v_month_end) by adding exactly one month to it. It dynamically constructs a formatted partition name string using the year and month (e.g., fct_orders_dd_2026_05). It also defines a temporary staging table name by appending _stage to the partition string (e.g., fct_orders_dd_2026_05_stage).

```
CREATE OR REPLACE PROCEDURE bl_cl.load_fct_orders_dd(
    p_month_start DATE
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows      INT := 0;
    v_month_end DATE := (p_month_start + INTERVAL '1 month')::DATE;
    v_part      TEXT := 'fct_orders_dd_' || TO_CHAR(p_month_start, 'YYYY_MM');
    v_stage     TEXT := v_part || '_stage';
```

If a partition for this month already exists, it detaches and drops it — making the load idempotent

```
BEGIN
    IF EXISTS (SELECT 1 FROM pg_inherits i
               JOIN pg_class c ON c.oid = i.inhrelid
               WHERE c.relname = v_part
                  AND i.inhparent = 'bl_dm.fct_orders_dd'::regclass) THEN
        EXECUTE format('ALTER TABLE bl_dm.fct_orders_dd DETACH PARTITION bl_dm.%I', v_part);
        EXECUTE format('DROP TABLE bl_dm.%I', v_part);
    END IF;
```

Building the month in a standalone table (same structure as the parent)

```
EXECUTE format('DROP TABLE IF EXISTS bl_dm.%I', v_stage);
EXECUTE format('CREATE TABLE bl_dm.%I (LIKE bl_dm.fct_orders_dd INCLUDING DEFAULTS)', v_stage);
```

Load the month (dynamic INSERT)

Reads ce_orders for just this month and LEFT JOINS each DM dimension on _src_id = 3NF surrogate::VARCHAR to translate 3NF surrogates into DM surrogate keys, defaulting misses to -1. It computes fct_net_amount on the fly and captures the row count.

```
EXECUTE format($f$
  INSERT INTO bl_dm.%I (
    order_surr_id, event_dt, product_surr_id, customer_surr_id,
    sales_rep_surr_id, city_surr_id, subchannel_surr_id,
    order_status, returned, payment_method,
    fct_unit_price, fct_quantity, fct_discount_percent, fct_discount_amount,
    fct_shipping_cost, fct_tax_amount, fct_order_amount, fct_cost_amount,
    fct_profit_margin_percent, fct_profit_amount, fct_net_amount,
    source_system, source_entity, order_src_id, insert_dt, update_dt)
  SELECT
    o.order_id,
    o.order_dt,
    COALESCE(dp.product_surr_id, -1),
    COALESCE(dc.customer_surr_id, -1),
    COALESCE(de.sales_rep_surr_id, -1),
    COALESCE(dl.city_surr_id, -1),
    COALESCE(dch.subchannel_surr_id, -1),
    o.order_status, o.returned, o.payment_method,
    o.unit_price, o.quantity, o.discount_percent, o.discount_amount,
    o.shipping_cost, o.tax_amount, o.order_amount, o.cost_amount,
    o.profit_margin_percent, o.profit_amount,
    (o.order_amount - o.discount_amount - o.shipping_cost - o.tax_amount),
    'BL_3NF', 'CE_ORDERS', o.order_src_id, CURRENT_DATE, CURRENT_DATE
  FROM bl_3nf.ce_orders o
  LEFT JOIN bl_dm.dim_products      dp ON dp.product_src_id      = o.product_id::VARCHAR
  LEFT JOIN bl_dm.dim_customers_scd dc ON dc.customer_src_id     = o.customer_id::VARCHAR
  LEFT JOIN bl_dm.dim_employees     de ON de.sales_rep_src_id    = o.sales_rep_id::VARCHAR
  LEFT JOIN bl_dm.dim_locations     dl ON dl.city_name_src_id    = o.city_id::VARCHAR
  LEFT JOIN bl_dm.dim_channels      dch ON dch.subchannel_src_id  = o.subchannel_id::VARCHAR
  WHERE o.order_dt >= %L AND o.order_dt < %L
$f$, v_stage, p_month_start, v_month_end);
GET DIAGNOSTICS v_rows = ROW_COUNT;
```

Rename + ATTACH

Renames the staging table to the partition name and runs ATTACH PARTITION.

```
-- Rename and ATTACH - this creates the partition automatically
EXECUTE format('ALTER TABLE bl_dm.%I RENAME TO %I', v_stage,
EXECUTE format($f$
  ALTER TABLE bl_dm.fct_orders_dd
  ATTACH PARTITION bl_dm.%I FOR VALUES FROM (%L) TO (%L)
$f$, v_part, p_month_start, v_month_end);
```

Logging and erroring

```
CALL bl_cl.p_log('load_fct_orders_dd', 'SUCCESS', v_rows,
  'Partition ' || v_part || ' auto-created & attached with ' || v_rows || ' rows');
EXCEPTION WHEN OTHERS THEN
  CALL bl_cl.p_log('load_fct_orders_dd', 'ERROR', NULL, SQLERRM);
END;
$$;
```

PROCEDURE bl_cl.load_fct_auto(p_window_months INT DEFAULT 3)

rolling-window driver

Anchor + empty guard

Reads MAX(order_dt) as the anchor (the latest month in the data, not today), and returns early with a log note if ce_orders is empty. This anchoring is why the window lands on real historical data instead of empty future months.

Discover + load months

Computes the window start, then queries ce_orders for the distinct months that actually have data inside the window, and calls load_fct_orders_dd for each. Nothing about the months is hardcoded - the driver discovers them from the data, so partitions are created purely in response to what's there.

Prune + log

Calls prune_fct_partitions to remove months that fell outside the window, then logs how many months were loaded. One call to load_fct_auto thus does the whole rolling-window cycle: create new partitions, drop old ones.

```
CREATE OR REPLACE PROCEDURE bl_cl.load_fct_auto(p_window_months INT DEFAULT 3)
LANGUAGE plpgsql
AS $$
DECLARE
    v_anchor      DATE;
    v_win_start   DATE;
    rec           RECORD;
    v_loaded      INT := 0;
BEGIN
    SELECT DATE_TRUNC('month', MAX(order_dt))::DATE INTO v_anchor
    FROM bl_3nf.ce_orders;

    IF v_anchor IS NULL THEN
        CALL bl_cl.p_log('load_fct_auto', 'SUCCESS', 0, 'No orders in ce_orders - nothing to load');
        RETURN;
    END IF;

    v_win_start := (v_anchor - ((p_window_months - 1) || ' month')::INTERVAL)::DATE;

    FOR rec IN
        SELECT DISTINCT DATE_TRUNC('month', order_dt)::DATE AS month_start
        FROM bl_3nf.ce_orders
        WHERE order_dt >= v_win_start
        ORDER BY 1
    LOOP
        CALL bl_cl.load_fct_orders_dd(rec.month_start);
        v_loaded := v_loaded + 1;
    END LOOP;

    CALL bl_cl.prune_fct_partitions(p_window_months);

    CALL bl_cl.p_log('load_fct_auto', 'SUCCESS', v_loaded,
        'Auto rolling-window: loaded ' || v_loaded ||
        ' month(s) from ' || v_win_start || ', pruned older');
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_fct_auto', 'ERROR', NULL, SQLERRM);
END;
$$;
```

PROCEDURE bl_cl.prune_fct_partitions(p_keep_months INT DEFAULT 3)

Anchor + cutoff

Reads MAX(order_dt) and computes the cutoff month - the boundary below which partitions are too old to keep. Anchoring on the data (not CURRENT_DATE) ensures the cutoff aligns with your historical dates.

Find + drop old partitions

Loops over the fact's partitions, parsing the month out of each partition's name, and for any older than the cutoff runs DETACH PARTITION then DROP TABLE. This removes whole months instantly with no table bloat, incrementing the dropped counter each time. A final p_log records how many were removed.

```
CREATE OR REPLACE PROCEDURE bl_cl.prune_fct_partitions(p_keep_months INT DEFAULT 3)
LANGUAGE plpgsql
AS $$
DECLARE
    v_anchor DATE;
    v_cutoff DATE;
    v_dropped INT := 0;
    rec RECORD;
BEGIN
    SELECT DATE_TRUNC('month', MAX(order_dt))::DATE INTO v_anchor
    FROM bl_3nf.ce_orders;
    v_cutoff := (v_anchor - ((p_keep_months - 1) || ' month')::INTERVAL)::DATE;

    FOR rec IN
        SELECT c.relname AS part_name
        FROM pg_inherits i
        JOIN pg_class c ON c.oid = i.inhrelid
        WHERE i.inhparent = 'bl_dm.fct_orders_dd'::regclass
            AND c.relname ~ '^fct_orders_dd\d{4}_\d{2}$'
            AND TO_DATE(RIGHT(c.relname, 7), 'YYYY_MM') < v_cutoff
    LOOP
        EXECUTE format('ALTER TABLE bl_dm.fct_orders_dd DETACH PARTITION bl_dm.%I', rec.part_name);
        EXECUTE format('DROP TABLE bl_dm.%I', rec.part_name);
        v_dropped := v_dropped + 1;
    END LOOP;
    CALL bl_cl.p_log('prune_fct_partitions', 'SUCCESS', v_dropped,
        'Detached/dropped ' || v_dropped || ' partition(s) older than ' || v_cutoff);
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('prune_fct_partitions', 'ERROR', NULL, SQLERRM);
END;
$;
```

TESTS

Logging shows that the procedures are rerunnable, with no duplicates

log_id	log_dt	AZ procedure_name	AZ log_status	rows_affected	AZ log_message
123	2026-07-25 13:16:51.659	load_fct_auto	SUCCESS	3	Auto rolling-window: loaded 3 month(s) from 2026-04-01, pruned older
122	2026-07-25 13:16:51.659	prune_fct_partitions	SUCCESS	0	Detached/dropped 0 partition(s) older than 2026-04-01
121	2026-07-25 13:16:51.659	load_fct_orders_dd	SUCCESS	14,257	Partition fct_orders_dd_2026_06 auto-created & attached with 14257 rows
120	2026-07-25 13:16:51.659	load_fct_orders_dd	SUCCESS	24,425	Partition fct_orders_dd_2026_05 auto-created & attached with 24425 rows
119	2026-07-25 13:16:51.659	load_fct_orders_dd	SUCCESS	23,781	Partition fct_orders_dd_2026_04 auto-created & attached with 23781 rows
118	2026-07-25 12:54:20.114	load_fct_auto	SUCCESS	3	Auto rolling-window: loaded 3 month(s) from 2026-04-01, pruned older
117	2026-07-25 12:54:20.114	prune_fct_partitions	SUCCESS	0	Detached/dropped 0 partition(s) older than 2026-04-01
116	2026-07-25 12:54:20.114	load_fct_orders_dd	SUCCESS	14,257	Partition fct_orders_dd_2026_06 auto-created & attached with 14257 rows
115	2026-07-25 12:54:20.114	load_fct_orders_dd	SUCCESS	24,425	Partition fct_orders_dd_2026_05 auto-created & attached with 24425 rows
114	2026-07-25 12:54:20.114	load_fct_orders_dd	SUCCESS	23,781	Partition fct_orders_dd_2026_04 auto-created & attached with 23781 rows

```
SELECT c.relname AS partition,
       pg_get_expr(c.relpartbound, c.oid) AS bounds,
       pg_size_pretty(pg_relation_size(c.oid)) AS size
FROM pg_inherits i JOIN pg_class c ON c.oid = i.inhrelid
WHERE i.inhparent = 'bl_dm.fct_orders_dd'::regclass ORDER BY 1;
```

ad_logs 1

pg_class 2

c.relname AS partition, pg_get_expr(c.relpartbound, c.oid) AS bounds, pg_size_pretty(pg_relation_size(c.oid)) AS size Enter a SQL expression to filter result

A-Z partition	A-Z bounds	A-Z size
fct_orders_dd_2026_04	FOR VALUES FROM ('2026-04-01') TO ('2026-05-01')	5280 kB
fct_orders_dd_2026_05	FOR VALUES FROM ('2026-05-01') TO ('2026-06-01')	5416 kB
fct_orders_dd_2026_06	FOR VALUES FROM ('2026-06-01') TO ('2026-07-01')	3160 kB
fct_orders_dd_default	DEFAULT	0 bytes